

复旦大学人工智能课程AI-Basics Courses

人工智能编程基础

Fundamentals of Artificial Intelligence Programming

计算机科学技术学院

周雅倩
zhuoyaqian@fudan.edu.cn

1

References

- <http://speech.ee.ntu.edu.tw/~hylee/ml/2023-spring.php>
 - Partial contents are copied from the slides of 李宏毅教授

2

卷积神经网络与计算机视觉
CNN and computer vision

卷积神经网络与计算机视觉

CNN and Computer Vision

- CNN基本思想
- CNN结构及经典网络
- 图像处理基础

3

卷积神经网络与计算机视觉
CNN and computer vision

卷积神经网络与计算机视觉

- 计算机视觉是一门多学科交叉的领域，涉及计算机科学、数学、物理学、生物学、心理学、神经科学等多个领域。它主要研究如何让机器或者计算机从数字图像或视频中获取并理解有用信息，实现对现实世界的感知和认知。
- 卷积神经网络（Convolutional Neural Network, CNN）是深度学习的一种特殊类型的神经网络，主要用于解决与图像相关的任务，如图像分类、目标检测、语义分割等。CNNs 是计算机视觉领域的核心技术之一，它们利用卷积运算来提取图像中的特征，并通过一系列层次化的抽象表示来理解图像内容。

4

卷积神经网络与计算机视觉
CNN and computer vision

CNN基本思想

卷积神经网络最初由Yann LeCun等人在1989年提出，是最初取得成功的深度神经网络之一。它的基本思想：

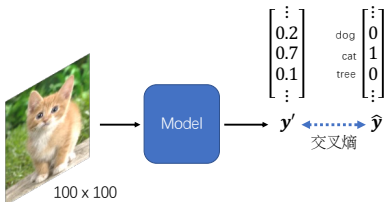
局部连接

参数共享

5

卷积神经网络与计算机视觉
CNN and computer vision

图像分类



100 x 100

(所有的分类图像都大小相同)

6

卷积神经网络与计算机视觉

CNN and computer vision

交叉熵

交叉熵损失函数经常用于分类问题中，特别是在神经网络做分类问题时，也经常使用交叉熵作为损失函数，交叉熵涉及到计算每个类别的概率，因此交叉熵几乎每次都和sigmoid(或softmax)函数一起出现。在图像分类中，经常使用softmax和交叉熵作为损失函数，交叉熵损失函数定义如下：

$$CrossEntropy = - \sum_{i=1}^n p(x_i) \ln(q(x_i)^{x_i})$$

其中， $p(x)$ 表示真实概率分布， $q(x)$ 表示预测概率分布。交叉熵损失函数通过缩小两个概率分布的差异，来使预测概率分布尽可能达到真实概率分布。

7

卷积神经网络与计算机视觉

CNN and computer vision

图像分类

3-D tensor

100 x 100

100

3个通道

值代表强度

8

卷积神经网络与计算机视觉

CNN and computer vision

灰度图像

灰度图像

灰度图像的表达方式

9

卷积神经网络与计算机视觉

CNN and computer vision

彩色图像

彩色图像的像素构成情况

10

卷积神经网络与计算机视觉

CNN and computer vision

观察1

Fully Connected Network

100 x 100

100 x 100

100 x 100 x 3

3 x 10³

1000

我们真的需要“完全连接”图像处理吗？

11

卷积神经网络与计算机视觉

CNN and computer vision

观察1

识别一些关键模式

Input

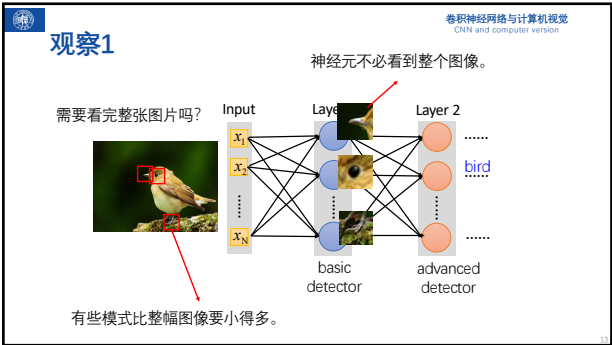
Layer

Layer 2

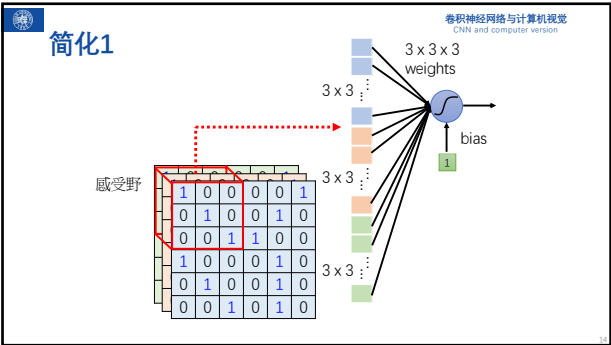
Bird?

也许人类也会以类似方式识别鸟类… ☺

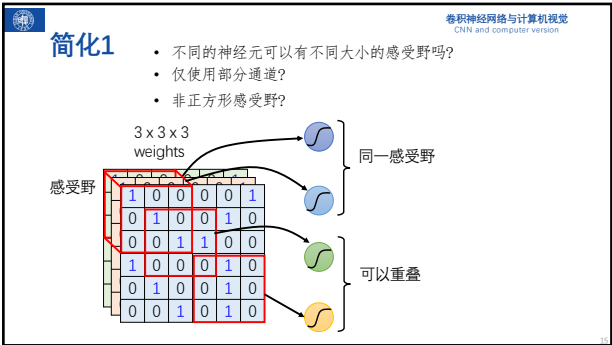
12



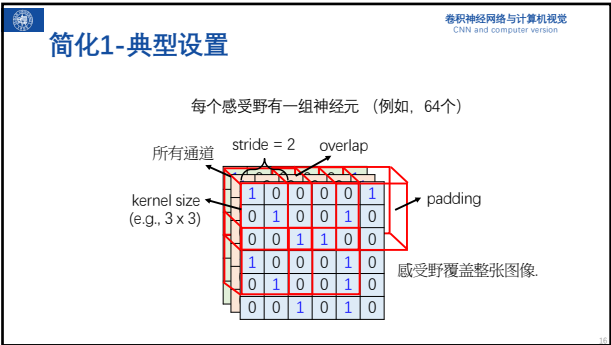
13



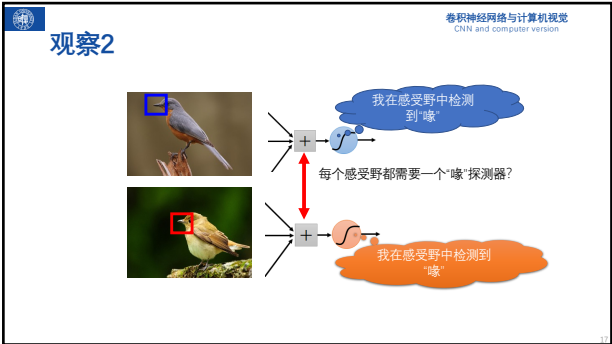
14



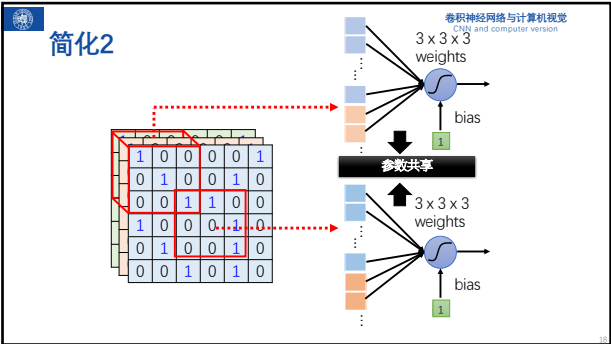
15



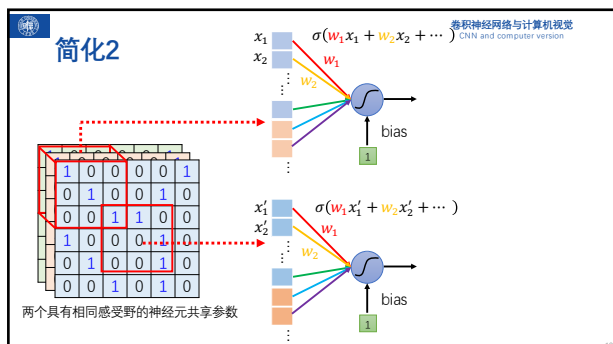
16



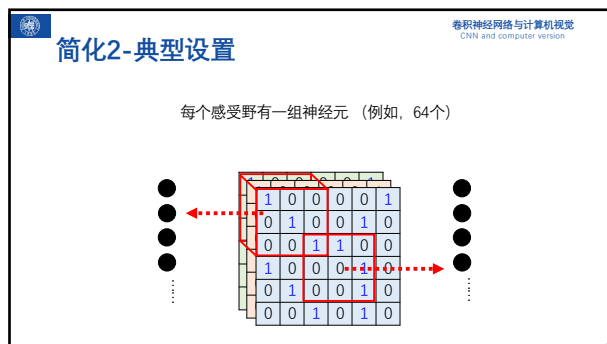
17



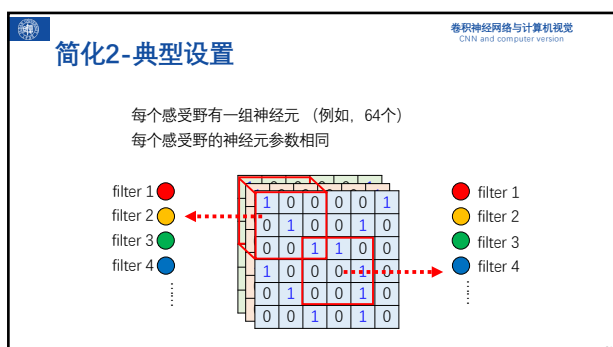
18



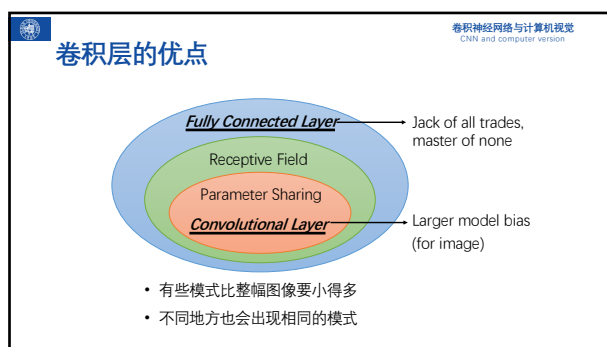
19



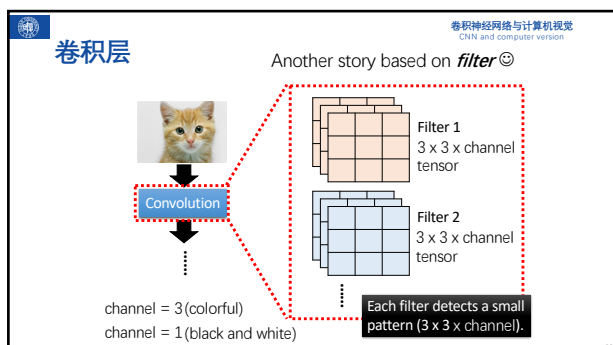
20



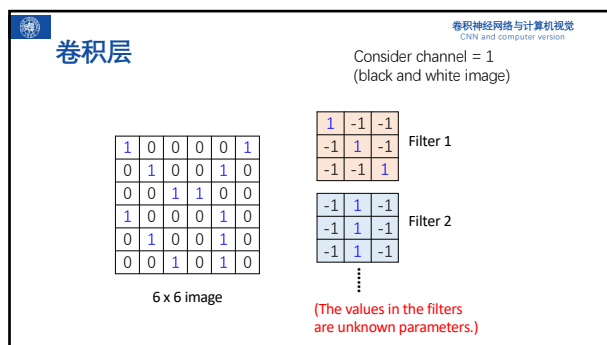
21



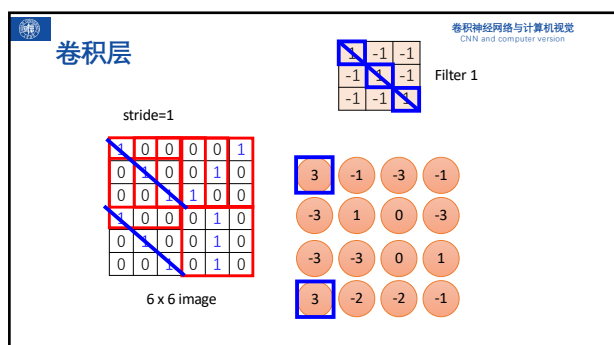
22



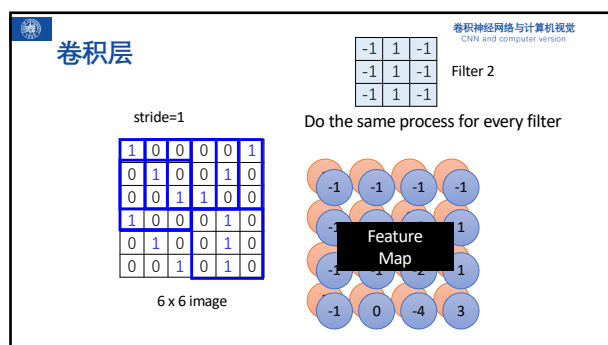
23



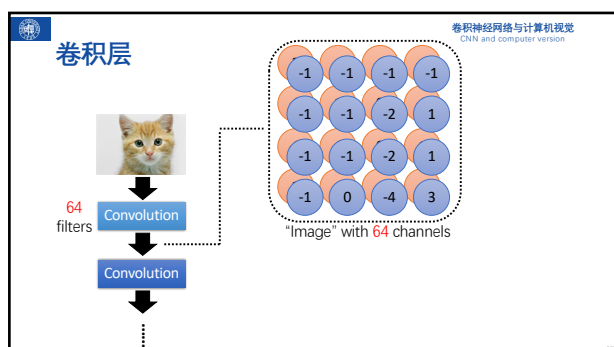
24



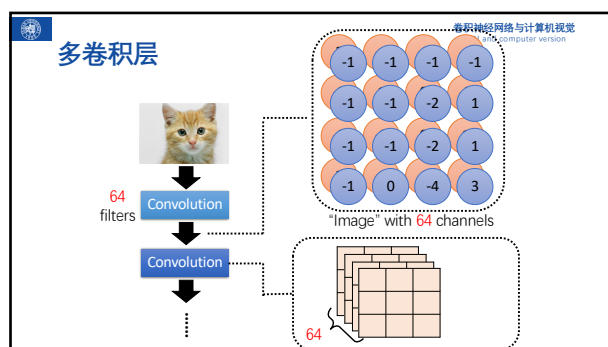
25



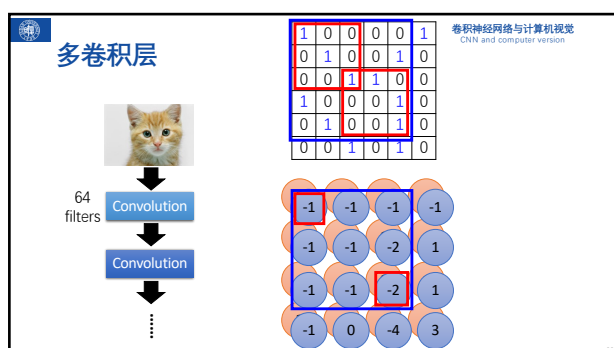
26



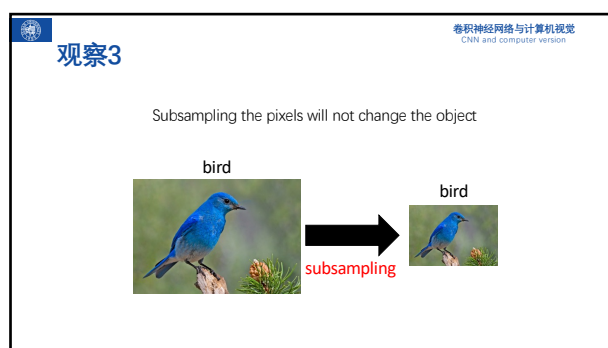
27



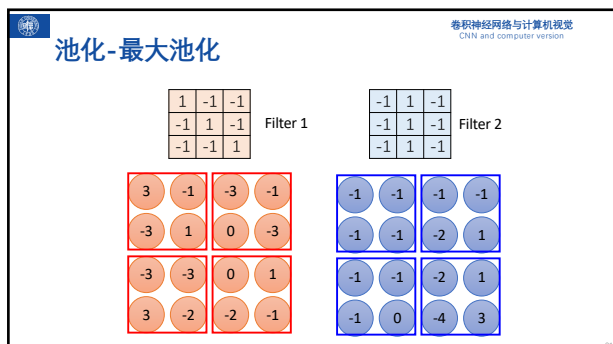
28



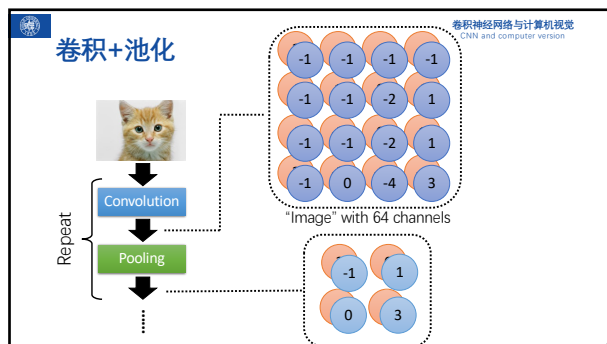
29



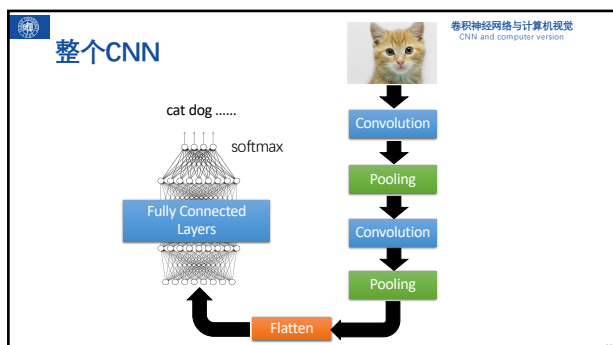
30



31



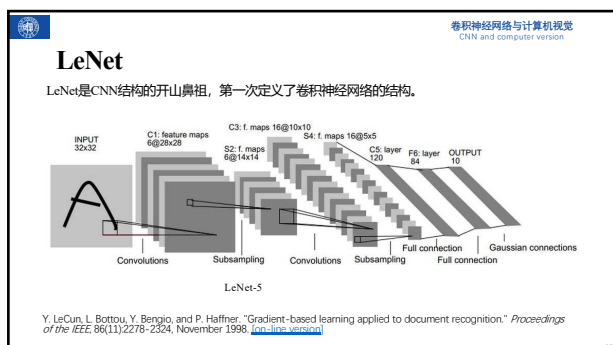
32



33



34



35



36

卷积神经网络与计算机视觉
CNN and computer vision

LeNet

```
def forward(self, x):
    batch_size = x.size(0)
    x = self.conv1(x) # 一层卷积层, 一层激活层, 一层池化层,
    x = self.conv2(x) # 再来一次
    x = self.conv3(x) # 再来一次
    x = x.view(batch_size, -1) # flatten 变成全连接网络需要的输入 (batch,
    120, 1, 1) ==> (batch, 120), -1 此处自动算出的是120
    x = self.fc(x)
    return x # 最后输出的是维度为10的, 也就是 (对应数字符号的0~9)
```

37

卷积神经网络与计算机视觉
CNN and computer vision

LeNet

```
from torchsummary import summary
# 查看模型的内存使用
summary(LeNet(), input_size=(1, 32, 32))
```

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 6, 28, 28]	156
ReLU-2	[-1, 6, 28, 28]	0
MaxPool2d-3	[-1, 6, 14, 14]	0
Conv2d-4	[-1, 16, 10, 10]	2,416
ReLU-5	[-1, 16, 10, 10]	0
MaxPool2d-6	[-1, 16, 5, 5]	0
Conv2d-7	[-1, 120, 1, 1]	48,120
ReLU-8	[-1, 120, 1, 1]	0
Linear-9	[-1, 84]	10,164
Linear-10	[-1, 10]	850

Total params: **61,706**
 Trainable params: 61,706
 Non-trainable params: 0
 Input size (MB): 0.00
 Forward/backward pass size (MB): 0.11
 Params size (MB): 0.24
 Estimated Total Size (MB): 0.35

38

卷积神经网络与计算机视觉
CNN and computer vision

全连接网络

```
class SimpleNN(nn.Module):
    def __init__(self):
        super(SimpleNN, self).__init__()
        self.fc1 = nn.Linear(784, 128)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(128, 64)
        self.fc3 = nn.Linear(64, 10)
```

Layer (type)	Output Shape	Param #
Linear-1	[-1, 128]	100,480
ReLU-2	[-1, 128]	0
Linear-3	[-1, 64]	8,256
ReLU-4	[-1, 64]	0
Linear-5	[-1, 10]	650

Total params: **109,386**
 Trainable params: 109,386
 Non-trainable params: 0
 Input size (MB): 0.00
 Forward/backward pass size (MB): 0.00
 Params size (MB): 0.42
 Estimated Total Size (MB): 0.42

```
from torchsummary import summary
# 查看模型的内存使用
summary(SimpleNN(), input_size=(28*28,))
```

39

卷积神经网络与计算机视觉
CNN and computer vision

数据准备

```
train_dataset = datasets.MNIST(root='./data', train=True, download=False,
transform=transform)
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)

test_dataset = datasets.MNIST(root='./data', train=False, download=False,
transform=transform)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)
```

40

卷积神经网络与计算机视觉
CNN and computer vision

数据预处理

```
transform = transforms.Compose([
    # 原始输入是一个一维张量 (784=28*28)
    # CNN的输入: 32行32列的图像, 这里转成二维的时候做了padding
    transforms.Resize((32, 32)),
    transforms.ToTensor(), # 转成张量形状为 [C, H, W], 并将像素值从 [0, 255] 缩放到 [0.0, 1.0]
    transforms.Normalize((0.1307, ), (0.3081, )) # 对图像数据进行归一化处理
])
```

41

卷积神经网络与计算机视觉
CNN and computer vision

模型训练

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
# 训练模型的函数
def train_model(model, train_loader, criterion, optimizer, num_epochs=10):
    model.train() # 模型设置为训练模式
    for epoch in range(num_epochs):
        for images, labels in train_loader:
            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()
```

42

模型测试

```
# 测试模型的函数
def evaluate_model(model, test_loader):
    model.eval() #模型设置为测试模式
    correct, total = 0, 0
    with torch.no_grad(): #禁用梯度
        for images, labels in test_loader:
            outputs = model(images)
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item() #统计分类正确的图像个数
    accuracy = 100 * correct / total #计算测试精度
    print(f'Accuracy of the network: {accuracy:.2f}%')
    return accuracy
```

43

CIFAR-10 数据集

➢ CIFAR-10 包含 60,000 张彩色图像，分为 10 个类别，每个类别有 6,000 张图像。
➢ 图像的分辨率为 32x32 像素，每个图像都有对应的标签（类别）。

```
# 定义数据处理
transform_train = transforms.Compose([
    transforms.RandomCrop(32, padding=4), #随机裁剪
    transforms.RandomHorizontalFlip(), #上下翻转
    transforms.ToTensor(),
    transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010)),
])
# 加载 CIFAR-10 数据集
trainset = torchvision.datasets.CIFAR10(root='./data', train=True, download=True,
transform=transform_train)
trainloader = DataLoader(trainset, batch_size=128, shuffle=True)
```

44

LeNet

- (1) 定义了卷积神经网络 (Convolutional Neural Network, CNN) 的基本框架：卷积层+池化层 (Pooling Layer) +全连接层；
- (2) 定义了卷积层 (Convolution Layer)，与全连接层相比，卷积层的不同之处有两点：局部连接 (引进“感受野”这一概念)、权值共享 (减少参数数量)；
- (3) 利用池化层进行下采样 (Downsampling)，从而减少计算量；
- (4) 用tanh作为非线性激活函数 (现在看到的都是改进过的LeNet了，用ReLU代替 tanh，相较于sigmoid, tanh以原点对称 (zero-centered)，收敛速度会快。

45

AlexNet

2012 年，Alex Krizhevsky, Ilya Sutskever 和 Geoffrey Hinton 推出了 AlexNet，引起了许多学者对深度学习的研究，可以算是深度学习的热潮的起始标志。

Model	Top-1 (val)	Top-5 (val)	Top-5 (test)
SIFT + FV4 [7]	—	—	26.2%
1 CNN	40.7%	18.2%	—
5 CNN*	38.1%	16.4%	16.4%
1 CNN*	39.0%	16.6%	—
7 CNN*	36.7%	15.4%	15.3%

Table 2: Comparison of error rates on ILSVRC-2012 validation and test sets. In *italics>* are best results achieved by others. Models with an asterisk* were “pre-trained” to classify the entire ImageNet 2011 Fall release. See Section 6 for details.

Krizhevsky, Alex, I. Sutskever, and G. Hinton. “ImageNet Classification with Deep Convolutional Neural Networks.” *Advances in neural information processing systems* 25.2(2012).

46

AlexNet

- (1) 采用双GPU网络结构，从而可以设计出更“大”、更“深”的网络（相较于当时的算力来说）；
- (2) 采用ReLU代替tanh，稍微解决梯度消失问题，加快网络收敛速度；
- (3) 提出局部响应归一化 (LRN, Local Response Normalization)；
- (4) 采用Overlapping Pooling，令其stride小于池化核的大小，从而使相邻的池化区域存在重叠部分；
- (5) 利用Dropout 避免网络过拟合；
- (6) 对训练图像做PCA（主成分分析），利用服从(0,0.1)的高斯分布的随机变量对主成分进行扰动；
- (7) 对训练数据进行随机裁剪 (Random Crop)，将训练图像由256×256裁剪为224×224，并做随机的镜像翻转 (Horizontal Reflection)，并在测试时，从图像的四个角以及中心进行裁剪，并进行镜像翻转，这样可以得到10个Patch，将这些 Patch 的结果进行平均，从而得到最终预测结果。

47

AlexNet 网络架构

知乎 @奥皮匠

48

卷积神经网络与计算机视觉
CNN and computer vision

Dropout层和BN层

- Dropout 是一种用于防止深度学习模型过拟合的正则化技术。由 Geoffrey Hinton 等人在 2012 年提出。
 - 它通过在训练过程中随机“丢弃”（即暂时移除）神经网络中的一部分神经元，从而减少神经元之间的复杂共适应关系，提高模型的泛化能力。
 - 例如：self dropout = nn.Dropout(0.5) # Dropout 层，概率为 0.5
- Batch Normalization (批量归一化，简称 BN) 是一种广泛使用的深度学习技术。它由 Sergey Ioffe 和 Christian Szegedy 在 2015 年提出，并迅速成为深度学习中的标准组件之一。
 - 核心思想是对使用当前批次的每一层的输入进行归一化处理，使其均值为 0，方差为 1，从而加速训练并提高模型的稳定性。
 - 例如：self.bn1 = nn.BatchNorm2d(64) # 批量归一化层，输入数据的通道数（对于 BatchNorm1d 是特征数）。

49

卷积神经网络与计算机视觉
CNN and computer vision

VGG

2014 年，Simonyan 和 Zisserman 提出了 VGG 系列模型（包括 VGG-11/VGG-13/VGG-16/VGG-19），并在当年的 ImageNet Challenge 上作为分类任务第二名、定位（Localization）任务第一名的基础网络出现。

VGG 与当时其他卷积神经网络不同，不采用感受野大的卷积核（如：7×7, 5×5），反而采用感受野小的卷积核（3×3）。关于这样做的好处有如下两点：

- 减少网络参数数量；由于参数数量被大幅减小，于是可以用多个感受野小的卷积层替换掉之前一个感受野大的卷积层，从而增加网络的非线性表达能力。

ConvNet Configuration					
A	A+LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv-3-64	conv-3-64	conv-3-64	conv-3-64	conv-3-64	conv-3-64
maxpool					
conv-3-128	conv-3-128	conv-3-128	conv-3-128	conv-3-128	conv-3-128
maxpool					
conv-3-256	conv-3-256	conv-3-256	conv-3-256	conv-3-256	conv-3-256
maxpool					
conv-3-512	conv-3-512	conv-3-512	conv-3-512	conv-3-512	conv-3-512
maxpool					
conv-3-512	conv-3-512	conv-3-512	conv-3-512	conv-3-512	conv-3-512
maxpool					
conv-3-512	conv-3-512	conv-3-512	conv-3-512	conv-3-512	conv-3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
self-max					

50

卷积神经网络与计算机视觉
CNN and computer vision

VGG

224 × 224 × 3 → 224 × 224 × 64 → 112 × 112 × 128 → 56 × 56 × 256 → 28 × 28 × 512 → 14 × 14 × 512 → 7 × 7 × 512 → 1 × 1 × 4096 → 1 × 1 × 1000

Legend:

- convolution + ReLU
- max pooling
- fully connected + ReLU
- softmax

51

卷积神经网络与计算机视觉
CNN and computer vision

InceptionNet

InceptionNet (GoogLeNet) 主要是由多个称为 Inception 模块实现的。它是一个分支结构，一共有 4 个分支。第一个分支是 1×1 卷积核，第二个分支是先进行 1×1 卷积，然后再 3×3 卷积。第三个分支同样先 1×1 卷积，然后再接一层 5×5 卷积。第四个分支是 3×3 的最大池化层，然后再用 1×1 卷积。最后，四个通道计算过的特征映射用沿通道维度拼接的方式组合到一起。

52

卷积神经网络与计算机视觉
CNN and computer vision

InceptionNet

GoogLeNet 是由 9 个 Inception v1 模块和 5 个池化层以及其他一些卷积层及全连接层组成，总共为 22 层网络。

53

卷积神经网络与计算机视觉
CNN and computer vision

ResNet

2015 年，Kaiming He 提出了 ResNet（拿到了 2016 年 CVPR Best Paper Award），不仅解决了神经网络中的退化问题还在同年的 ILSVRC 和 COCO 竞赛横扫竞争对手，分别拿下分类、定位、检测、分割任务的第一名。

Kaiming 在文中提出了残差结构（Residual Block），使得原本所要拟合的函数 $H(x)$ 改为 $F(x)$ ，其中 $H(x) = F(x) + x$ 。虽然在“多个非线性层可以拟合任意函数”这一假设下二者并无区别，但是 Kaiming 假设模型学习后者，将更容易进行优化与收敛。（在残差结构中，模型利用 Shortcut 进行 Identity Mapping，这样也解决了梯度消失现象）。

54

为什么在计算机视觉中使用CNN

1. **局部感知**: CNN 中的卷积层能够对输入数据进行局部感知, 这意味着每个神经元只与输入图像的一小部分相连。这种设计符合自然图像中物体和特征通常由局部相关像素构成的特点。例如, 在一张图片中识别一只猫的脸时, 边缘、眼睛等局部特征对于识别至关重要。

2. **参数共享**: 在卷积层中, 同一层的所有神经元都使用相同的权重 (即卷积核/滤波器)。这减少了模型的参数数量, 从而降低了过拟合的风险, 并且提高了计算效率。同时, 这也使得网络能够学习到平移不变性, 即无论特征出现在图像中的哪个位置, 网络都能识别它。

3. **层次化的特征提取**: CNN 通过多层卷积操作可以自动地从低级到高级抽象出图像的不同层面特征。初始层可能会检测到简单的边缘或颜色信息, 而更深层则可能组合这些简单特征来表示更加复杂的结构, 如形状或者纹理。

4. **池化操作**: 池化层通常跟在卷积层之后, 用于降低特征图的空间维度。这样做不仅可以减少后续处理的数据量, 提高计算效率, 还能够在一定程度上实现空间上的平移不变性。

5. **强大的表征能力**: 随着深度的增加, CNN 可以学习到非常丰富的图像表征, 这对于复杂的视觉任务是必需的。现代的 CNN 架构, 比如 ResNet、Inception 和 VGG 等, 已经在各种基准测试中取得了卓越的表现。

6. **端到端的学习**: 与传统方法相比, 如手工设计的特征提取加上分类器, CNN 能够直接从原始像素数据中学习有用的特征并进行预测, 不需要人工干预。这种方法大大简化了开发流程, 并允许算法从数据中自动学习最佳的特征表示。

7. **适应多种视觉任务**: 除了图像分类外, CNN 还能被灵活地应用于其他视觉任务, 比如目标检测、语义分割、图像生成等, 只需要调整网络架构即可。

55

图像基础：数字图像

01 **数字图像**, 又称数码图像或数位图像, 是由模拟图像数字化得到的、以像素 (或像元, Pixel) 为基本元素, 可以用计算机或数字电路存储和处理的图像。

02 数字图像可以由不同的输入设备和技术生成, 例如数码相机、扫描仪、坐标测量机等, 也可以从任意的非图像数据合成得到, 例如用数学函数、三维几何模型等得到。

56

数字图像

01 像素 (或像元, Pixel) 是在模拟图像数字化时对连续空间进行离散化得到的。每个像素具有整数行 (高) 和列 (宽) 位置坐标, 同时每个像素都具有整数灰度值或颜色值。

02 通常像素在计算机中保存为二维整数数组的光栅图像, 这些值常用压缩格式进行传输和储存。

03 我们可以这样理解数字图像, 图像是由许多像素组成的, 图像处理是在二维平面上对图片的每一个像素进行处理, 而如何知道我们处理的是这一个像素, 而不是另外一个像素, 就需要定义一个图像的最基本的元素, 这个元素就是像素。

57

数字图像数字化表示

二值图像是指仅仅包含黑色和白色两种颜色的图像, 图像的每个像素只能是黑或者白, 其像素值为0或1。下图为二值图像。

二值图像

58

灰度图像

01 一幅灰度图像就是一个数据矩阵, 矩阵的值表示灰度的浓淡。其每个像素只有一个采样颜色的图像, 这类图像通常显示为从最暗黑色到最亮的白色。

02 灰度图像与黑白图像不同, 在计算机图像领域中黑白图像只有黑色与白色两种颜色; 灰度图像在黑色与白色之间还有许多级的颜色深度。

03 灰度图像经常是在单个电磁波频谱如可见光内测量每个像素的亮度得到的。用于显示的灰度图像通常用每个采样像素8位 (uint8) 的非线性尺度来保存, 这样可以有256级灰度 (如果用16 (uint16) 位, 则有65536级)。

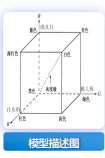


59

彩色图像

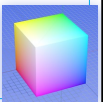
RGB颜色模型

- RGB模型也称为加色法混色模型。它是以R、G、B三原色光互相叠加来实现混色的方法, 适合于显示器等发光体的显示。
- 人类视觉系统能感知的颜色都可以用红、绿、蓝三种基色光按照不同的比例混合, 例如: 白色=100%红色+100%绿色+100%蓝色; 黄色=100%红色+100%绿色+0%蓝色等。



模型描述图

在RGB彩色模型中表示的图像由R、G、B三个8比特分量图像构成, 三幅图像在屏幕上混合生成一幅合成的24比特彩色图像, 也即是全彩色图像, 如图对应的24位彩色立方图可表示为:



60

彩色图像

HSV模型

HSV颜色空间是一种用色调 (H)、饱和度 (S)、亮度 (V) 联合表示的颜色模型。其色彩空间由3部分组成:

Hue(色调):
颜色种类, 如红、蓝、黄, 范围 $0 \sim 360^\circ$, 每个值对应着一种颜色。

Value(亮度):
颜色的亮度, 范围 $0 \sim 100\%$, 0总是黑色, 100时根据饱和度, 可能为白色或饱和度更低的颜色。

Saturation(饱和度):
颜色的纯度是颜色的丰满程度, 范围 $0 \sim 100\%$, 0表示没有颜色, 100表示饱和的颜色, 降低饱和度其实际就是在颜色中增加灰色的分量。

由于H、S分量代表了色彩信息, 不同HS值在表示颜色时有较大差异, 所以该模型可用于颜色分割。模型可表示为:



61

图像文件格式

BMP (Bitmap) 格式
GIF (Graphics Interchange Format) 格式
TIFF (Tagged Image File) 格式
JPEG (Joint Photographic Experts Group) 格式

62

激活函数

激活层的作用在于将前一层的线性输出, 通过非线性的激活函数进行处理, 这样用以模拟任意函数, 从而增强网络的表征能力。

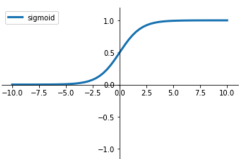
激活函数是一些非线性的函数, 这些函数的特性有所不同:

1. sigmoid函数可以将数值压缩到 $[0, 1]$ 的区间。
2. tanh可以将数值压缩到 $[-1, 1]$ 的区间。
3. ReLU函数实现一个取正的效果, 所有负数的信息都抛弃。
4. LeakyReLU是一种相对折中的ReLU, 认为当数值为负的时候可能也存在有一定有用的信息, 那么就乘以一个系数0.1 (可以调整或自动学习), 从而获取负数中的一部分信息。
5. Maxout使用两套参数, 取其中值大的一套作为输出。
6. ELU类似于LeakyReLU, 只是使用的公式不同。

63

sigmoid函数

在逻辑回归中使用sigmoid函数, 该函数是将取值为 $(-\infty, +\infty)$ 的数映射到 $(0, 1)$ 之间, sigmoid函数的公式以及图形如图所示

$$f(z) = \frac{1}{1+e^{-z}}$$


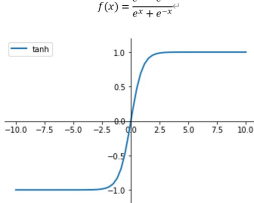
- 1) 当输入稍微远离了坐标原点, 函数的梯度就变得很小了, 几乎为零。在神经网络反向传播的过程中, 我们都是通过微分的链式法则来计算各个权重 w 的微分。当反向传播经过了sigmoid函数, 这个链条上的微分就很小了, 况且还可能经过很多个sigmoid函数, 最后会导致权重 w 对损失函数几乎没有影响, 这样不利于权重的优化, 这个问题叫做梯度饱和, 也可以叫梯度弥散。
- 2) 函数输出不是以0为中心的, 这样会使权重更新效率降低。
- 3) sigmoid函数要进行指数运算, 这个对于计算机来说是比较慢的。

64

tanh函数

导数为: $1 - \tanh^2(x)$

tanh函数相较于sigmoid函数要常见一些, 该函数是将取值为 $(-\infty, +\infty)$ 的数映射到 $(-1, 1)$ 之间, 其公式与图形如图所示。

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$


tanh函数在0附近很短一段区域内可看做线性的。由于tanh函数均值为0, 因此弥补了sigmoid函数均值为0.5的缺点。

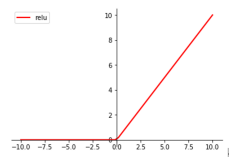
tanh函数的缺点同sigmoid函数的第一个缺点一样, 当 x 很大或很小时, $g'(x)$ 接近于0, 会导致梯度很小, 权重更新非常缓慢, 即梯度消失问题。

一般二分类问题中, 隐藏层用tanh函数, 输出层用sigmoid函数。不过这些也都不是一成不变的, 具体使用什么激活函数, 还是要根据具体的问题来具体分析, 还是要靠调试的。

65

ReLU函数

ReLU函数又称为修正线性单元 (Rectified Linear Unit), 是一种分段线性函数, 其弥补了sigmoid函数以及tanh函数的梯度消失问题。ReLU函数的公式以及图形:

$$f(x) = \begin{cases} x, & \text{if } x \geq 0 \\ 0, & \text{if } x < 0 \end{cases}$$


优点:

- 1) 在输入为正数的时候, 不存在梯度饱和问题。
- 2) 计算速度要快很多。ReLU函数只有线性关系, 不管是前向传播还是反向传播, 都比sigmoid和tanh要快很多。(sigmoid和tanh要计算指数, 计算速度会比较慢)

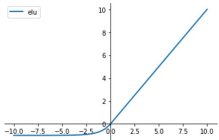
缺点:

- 1) 当输入是负数的时候, ReLU是完全不被激活的, 这就表明一旦输入到了负数, ReLU就会死掉。这样在前向传播过程中, 还不算什么问题, 有的区域是敏感的, 有的是不敏感的。但是到了反向传播过程中, 输入负数, 梯度就会完全到0, 这个和sigmoid函数、tanh函数有一样的问题。
- 2) ReLU函数也不是以0为中心的函数。

66

ELU函数

ELU函数是针对ReLU函数的一个改进型，相比于ReLU函数，在输入为负数的情况下，是有一定的输出的，而且这部分输出还具有一定的抗干扰能力。这样可以消除ReLU死掉的问题，不过还是有梯度饱和和指数运算的问题。

$$f(x) = \begin{cases} x, & x \geq 0 \\ a(e^x - 1), & x < 0 \end{cases}$$


67

损失函数

在深度学习中，损失函数扮演着至关重要的角色。通过对最小化损失函数，使模型达到收敛状态，减少模型预测值的误差。因此，不同的损失函数，对模型的影响是重大的。经常用到的下列损失函数：

- 图像分类：交叉熵
- 目标检测：Focal loss, L1/L2损失函数, IOU Loss, GIoU, DIOU, CIOU
- 图像识别：Triplet Loss, Center Loss, Sphereface, Cosface, Arcface

68

LogSoftmax 函数 $\ln(q(x))$ 的计算

Softmax 函数
给定一个向量 $\mathbf{z}=[z_1, z_2, \dots, z_n]$ ，Softmax 函数定义为：

$$\text{Softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

LogSoftmax 函数
LogSoftmax 函数则是将 Softmax 的结果取对数。对于向量 $\mathbf{z}$ ，LogSoftmax 定义为：

$$\text{LogSoftmax}(\mathbf{z})_i = \log \left(\frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}} \right)$$

这可以进一步简化为：

$$\text{LogSoftmax}(\mathbf{z})_i = z_i - \log \left(\sum_{j=1}^n e^{z_j} \right)$$

69

L1、L2、smooth L1损失函数

L1范数损失函数，也被称为最小绝对值偏差 (LAD)，最小绝对值误差 (LAE)。它是把目标值 (Y_i) 与估计值 ($\hat{f}(x_i)$) 的绝对差值的总和S最小化，定义如下：

$$S = \sum_{i=1}^n |Y_i - f(x_i)|$$

L2范数损失函数，也被称为最小平方误差 (LSE)。它是把目标值(Y_i)与估计值($\hat{f}(x_i)$)的差值的平方和(S)最小化，定义如下：

$$S = \sum_{i=1}^n (Y_i - f(x_i))^2$$

smooth L1损失函数，是光滑之后的L1，改善L1的有折点，不光滑，导致不稳定等缺点，smooth L1损失函数为，定义如下：

$$\text{smooth } \hat{L}_{L1}(x) = \begin{cases} 0.5x^2, & |x| < 1 \\ |x| - 0.5, & \text{otherwise} \end{cases}$$

70

L1、L2、smooth L1损失函数

从损失函数对x的导数可知：

- L1损失函数对x的导数为常数，在训练后期，x很小时，如果learning rate 不变，损失函数会在稳定值附近波动，很难收敛到更高的精度。
- L2损失函数对x的导数在x值很大时，其导数也非常大，在训练初期不稳定。
- smooth L1完美的避开了L1和L2损失的缺点。

在一般的目标检测中，通常是计算4个坐标值与Ground Truth框(已知的输入和输出)之间的差异，然后将这4个loss进行相加，构成regression loss。

71